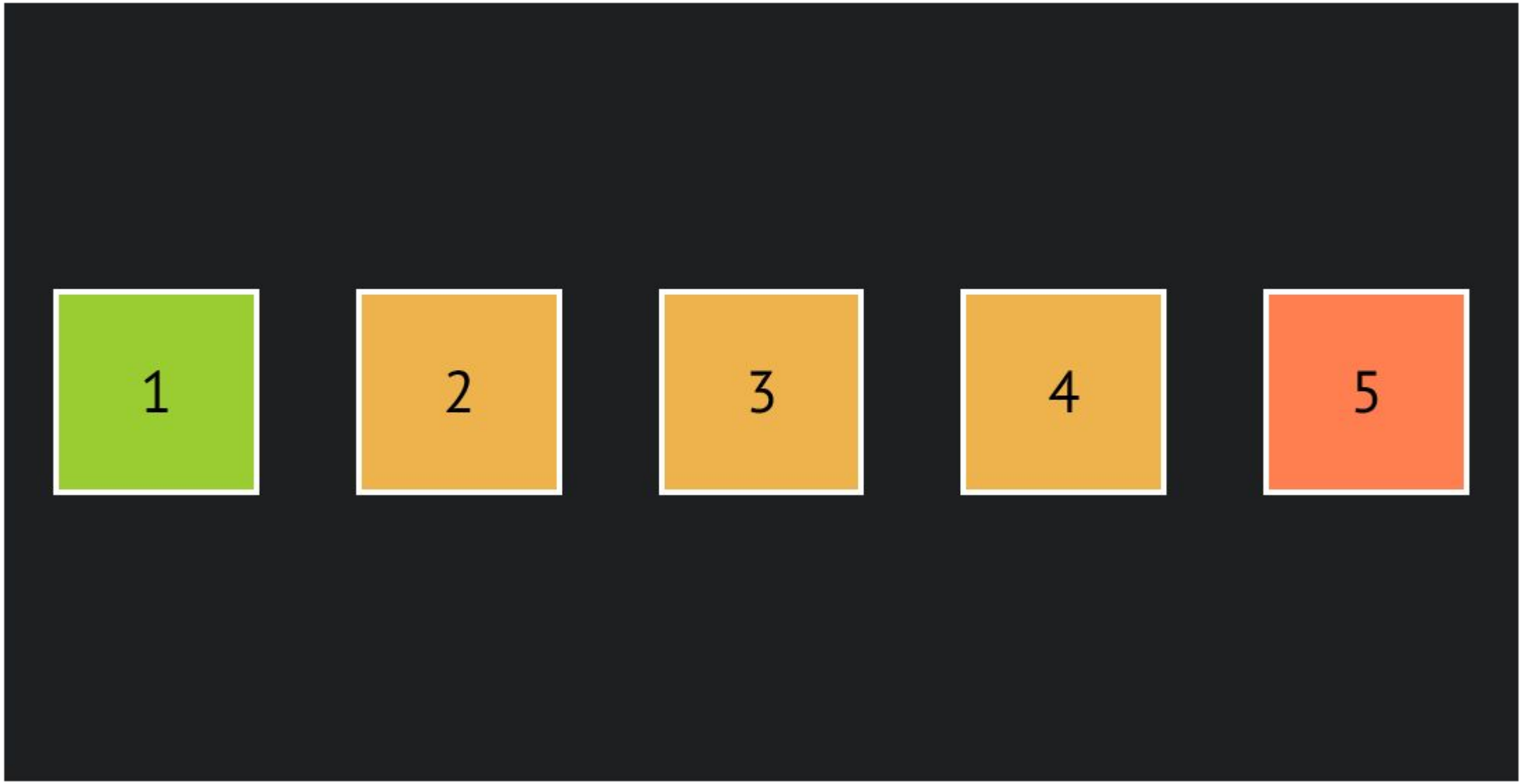
Flexbox

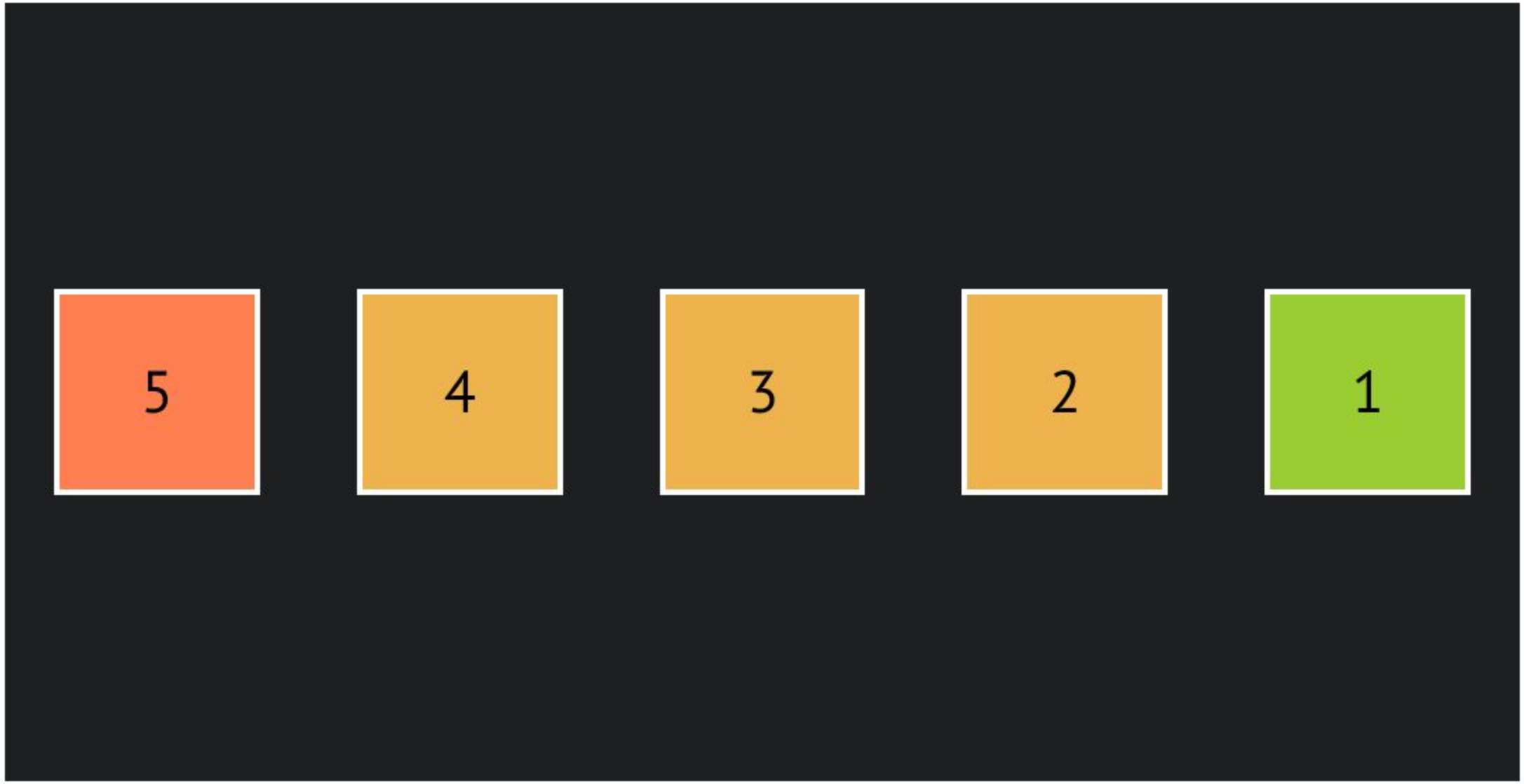
Flexbox

Главная задумка flex-вёрстки в наделении контейнера способностью изменять ширину/высоту (и порядок) своих элементов для наилучшего заполнения пространства (в большинстве случаев — для поддержки всех видов дисплеев и размеров экранов).
Flex-контейнер растягивает элементы для заполнения свободного места или сжимает их, чтобы предотвратить выход за границы.

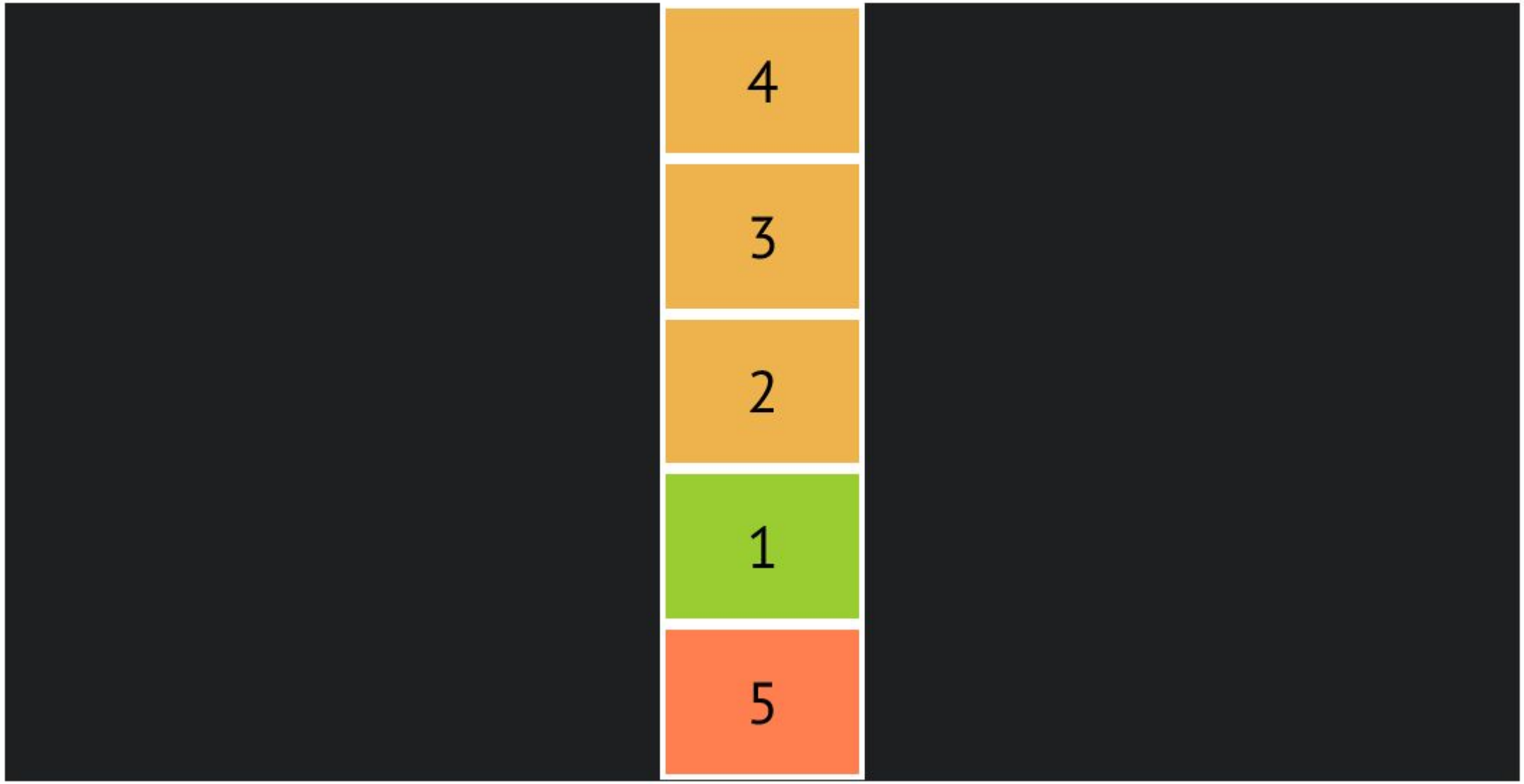
Flexbox умеет так



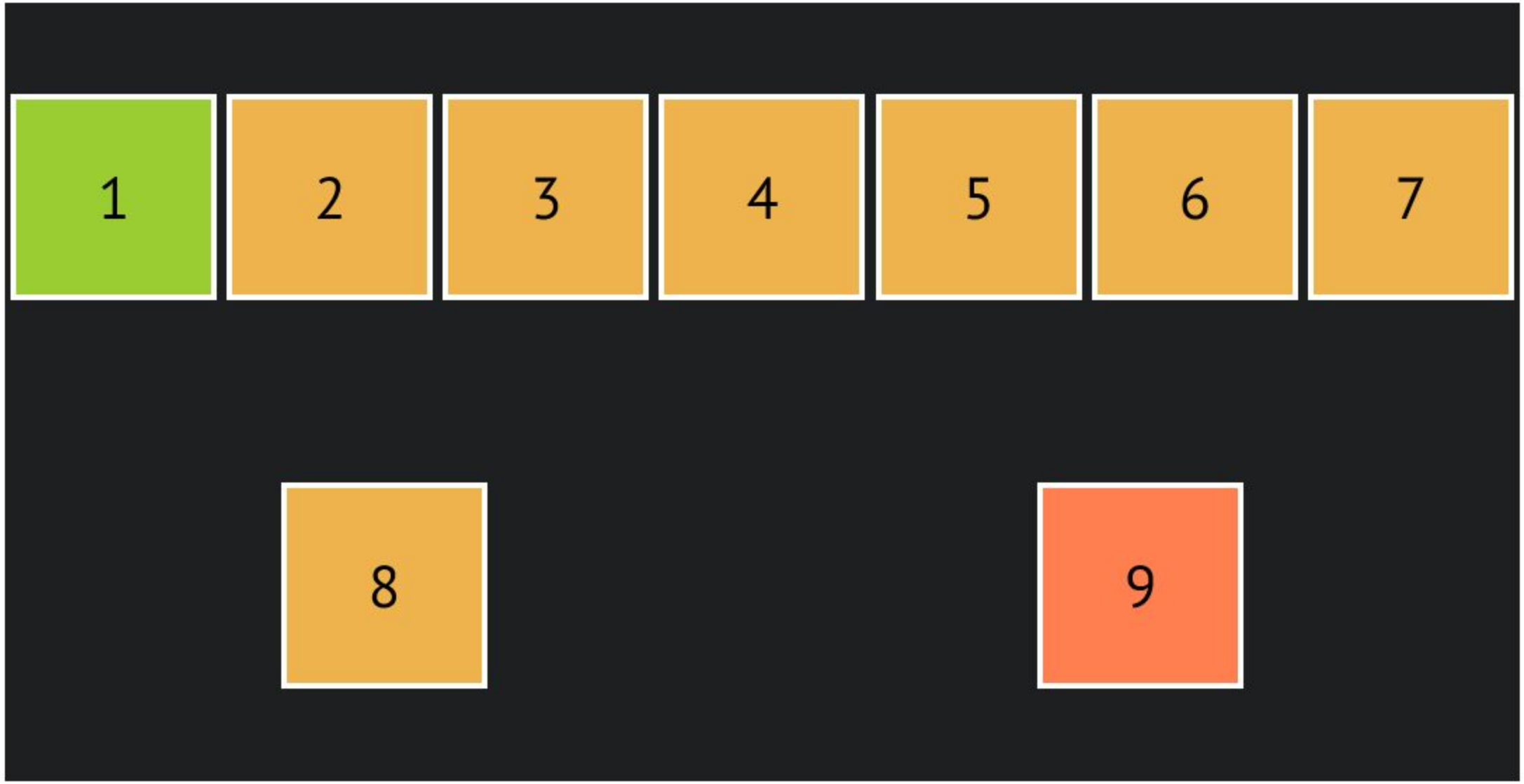
И так



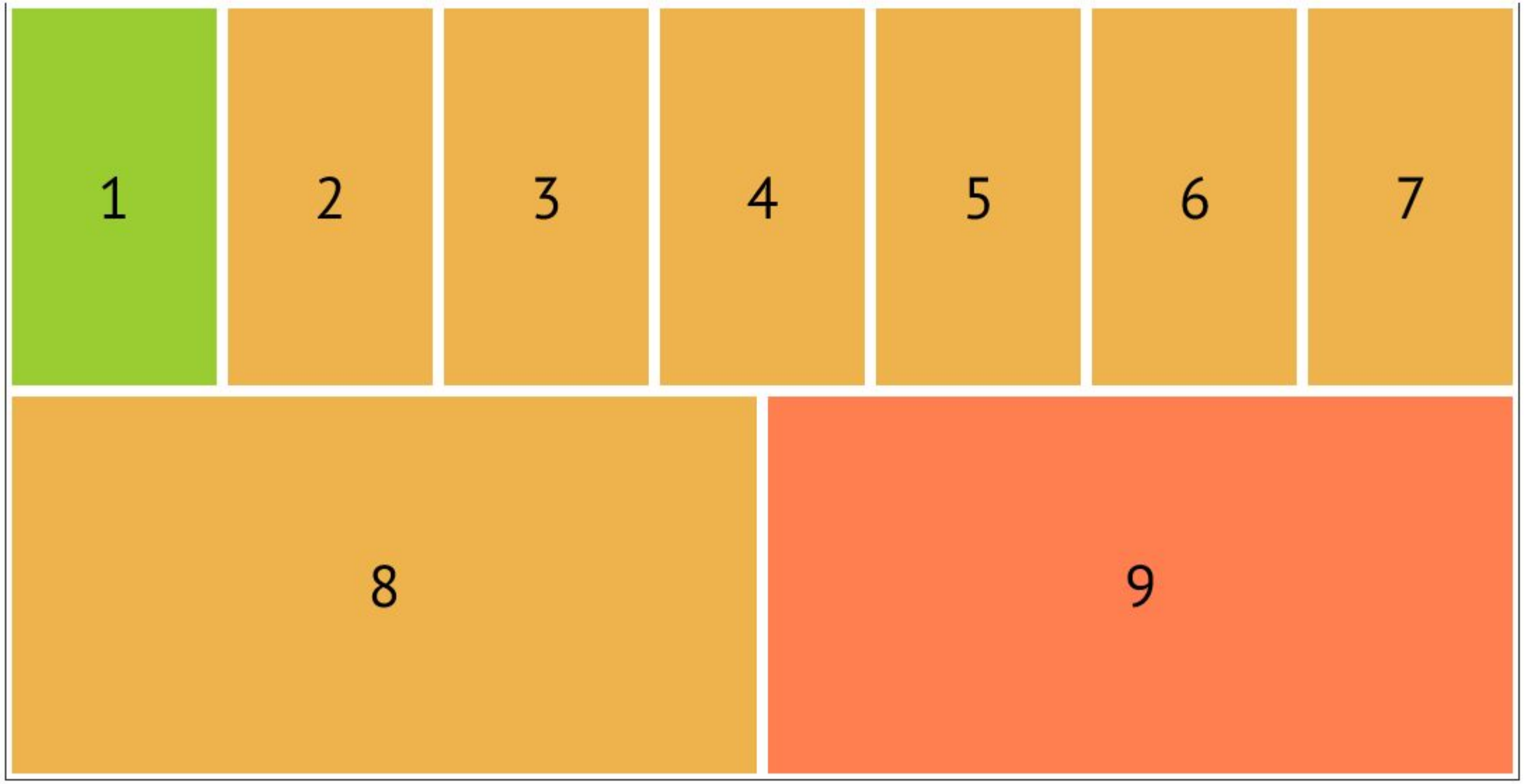
И вот так



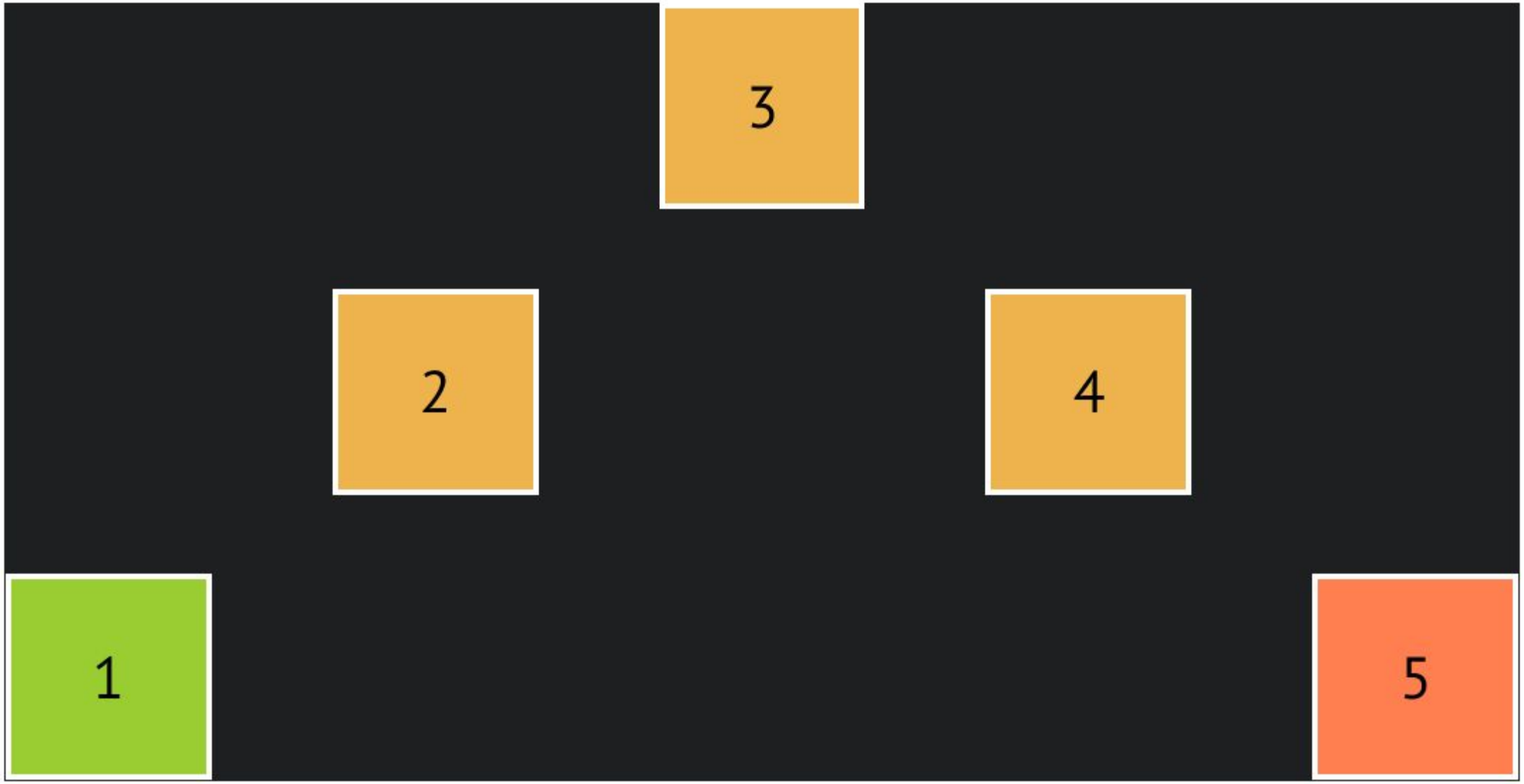
И этак



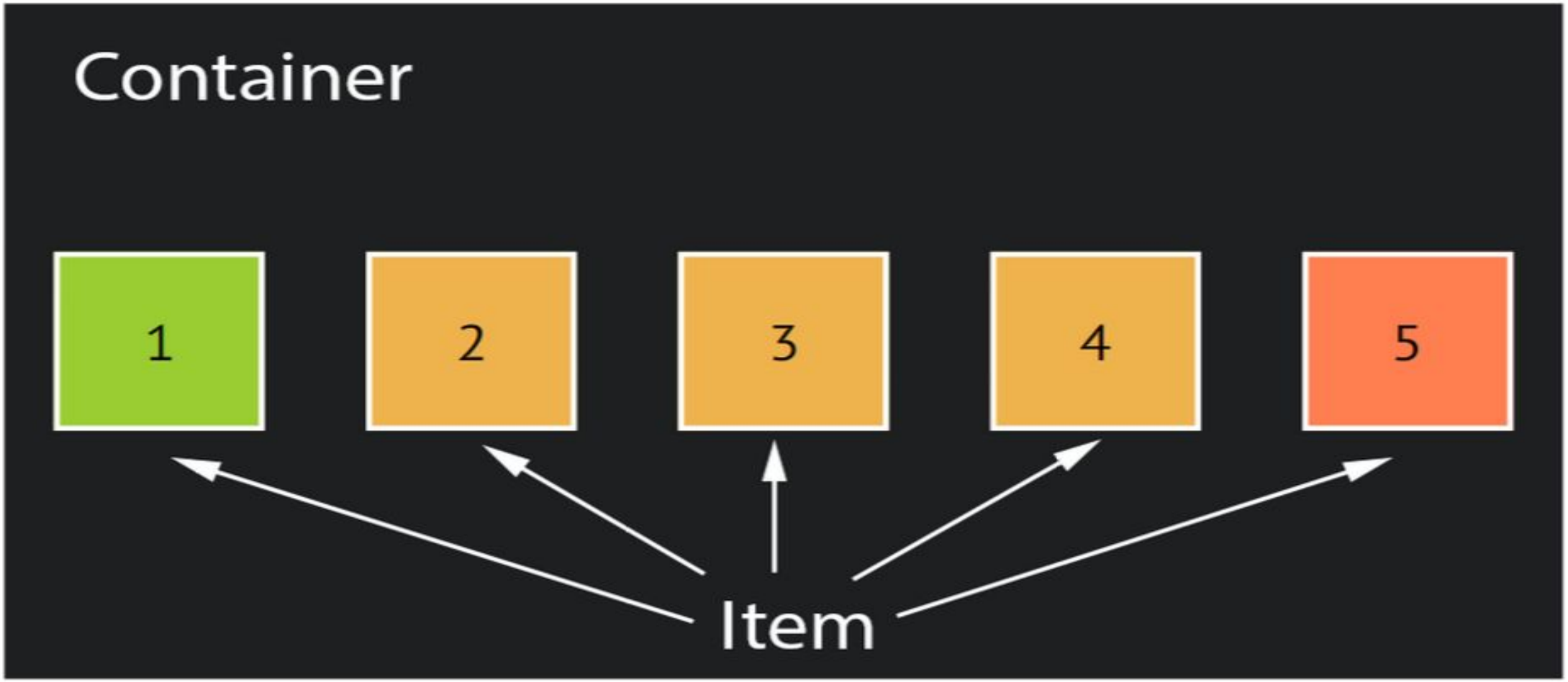
И вот этак



И, при желании, даже так

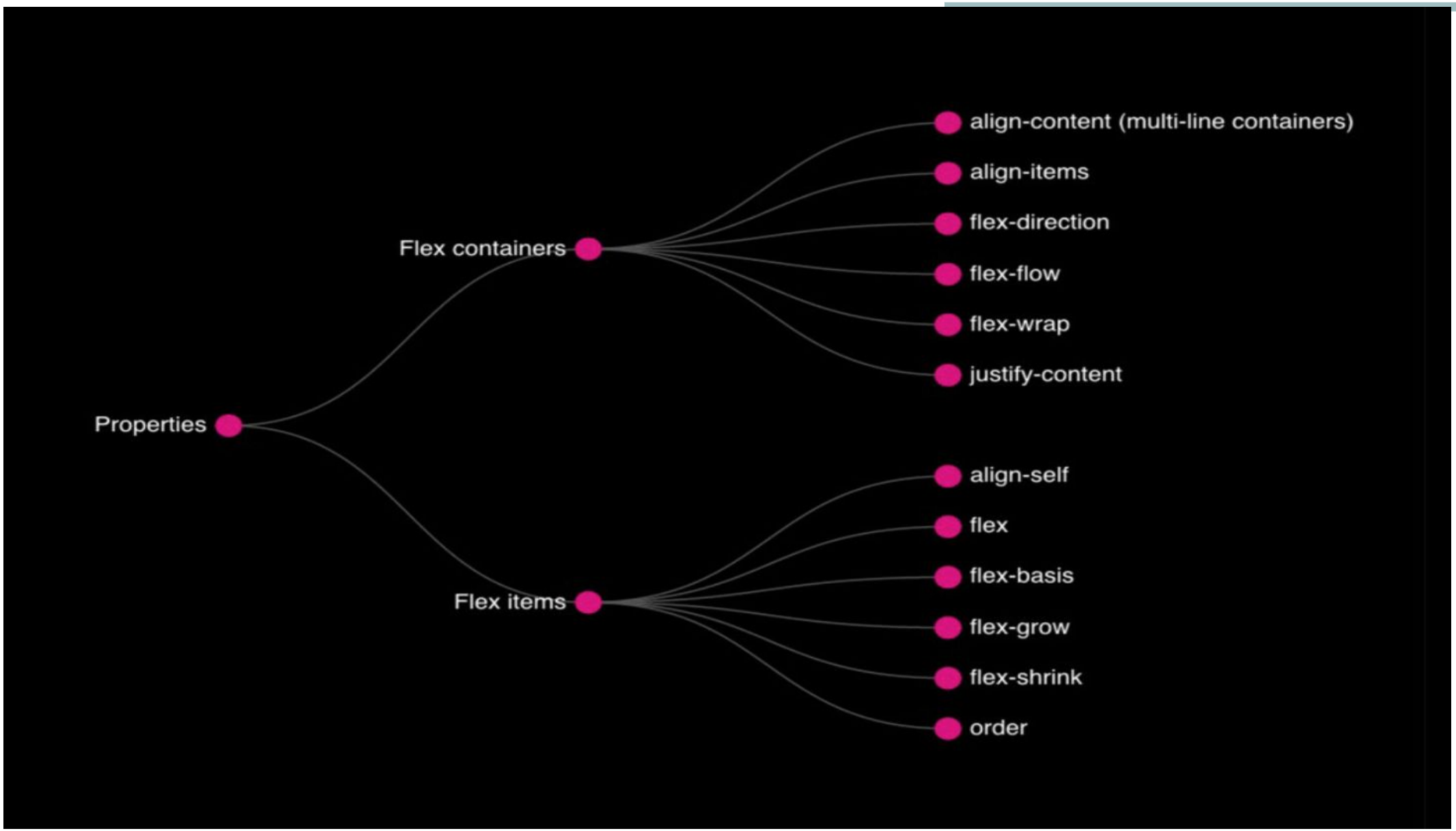


Субъекты



HTML

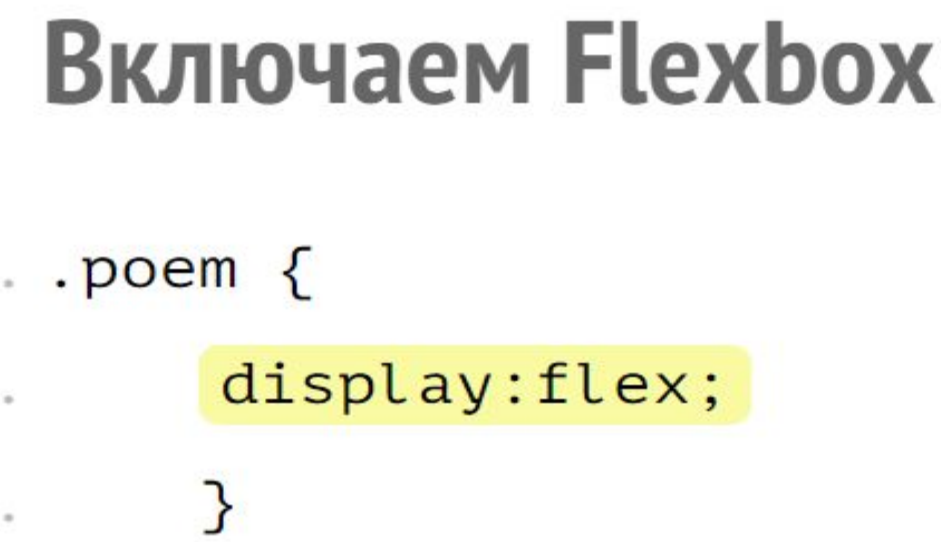
```
01. <div class="container">
02.     <div class="item">1</div>
03.     <div class="item">2</div>
04.     <div class="item">3</div>
05.     <div class="item">4</div>
06.     <div class="item">5</div>
07. </div>
```


Properties
Flex containers
align-content (multi-line containers)
align-items
flex-direction
flex-flow
flex-wrap
justify-content
Flex items
align-self
flex
flex-basis
flex-grow
flex-shrink
order

display: flex | inline-flex;

Применяется к:
родительскому элементу flex-контейнера.

Определяет flex-контейнер (инлайновый или блочный в зависимости от выбранного значения), подключает flex-контекст для всех его непосредственных потомков.



По умолчанию

- flex-элементы во flex-контейнере занимают всю его высоту.
- ширина элемента= содержимому элемента
- если задана ширина у элемента, то элемент занимает место по содержимому контейнера, но не больше своей ширины.
- может сжиматься
- не может растягиваться
- flex-элементы не переносятся на новую линию, даже когда им не хватает места в текущей линии. Они просто выходят за её пределы.

Включим flexbox

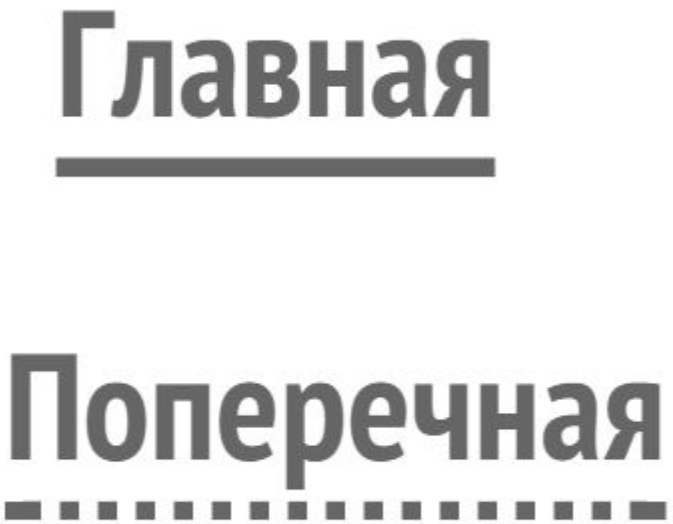


HTML

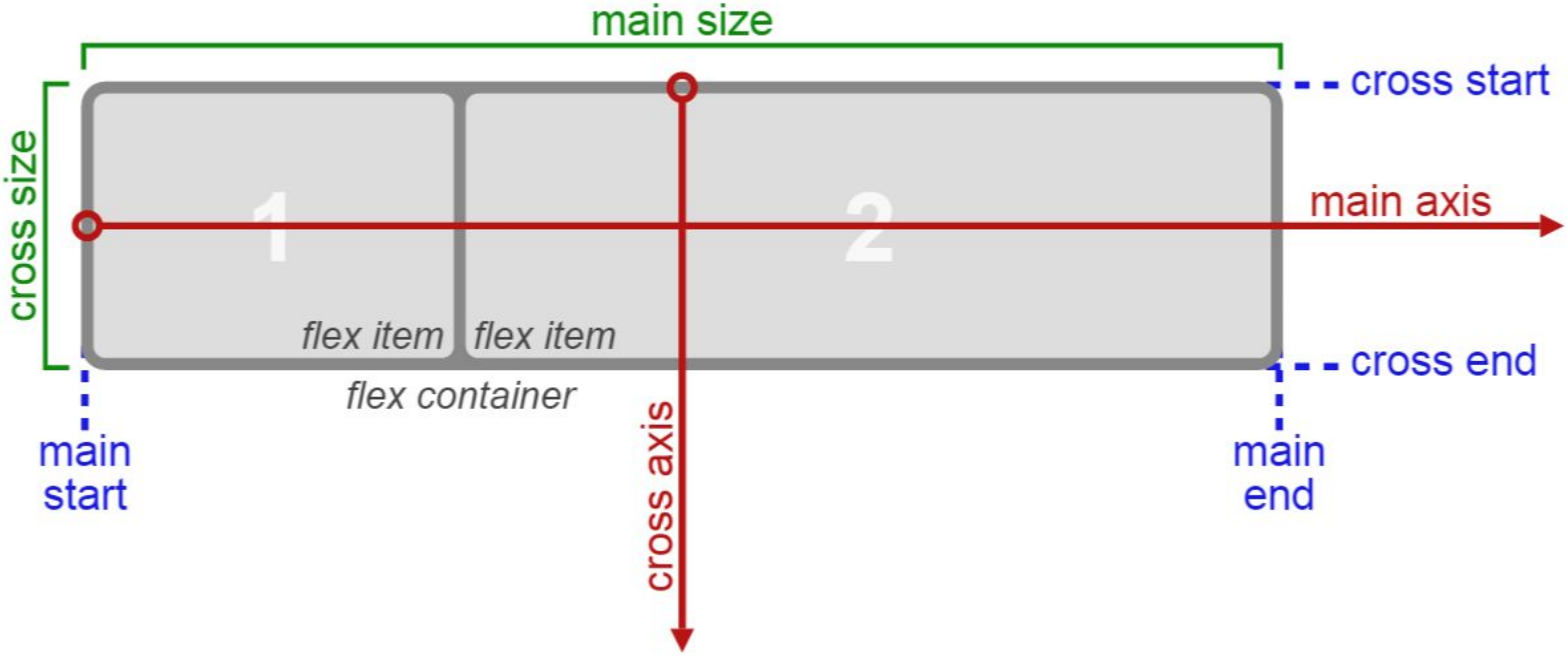
```
<div class=“flex">
    <div><h1>Я ПЕРВЫЙ</h1></div>
    <div><h1>Я ВТОРОЙ</h1></div>
    <div><h1>Я ТРЕТИЙ</h1></div>
    <div><h1>Я ЧЕТВЕРТЫЙ</h1></div>
</div>
```

```
.flex {
        display:flex;
    }
```

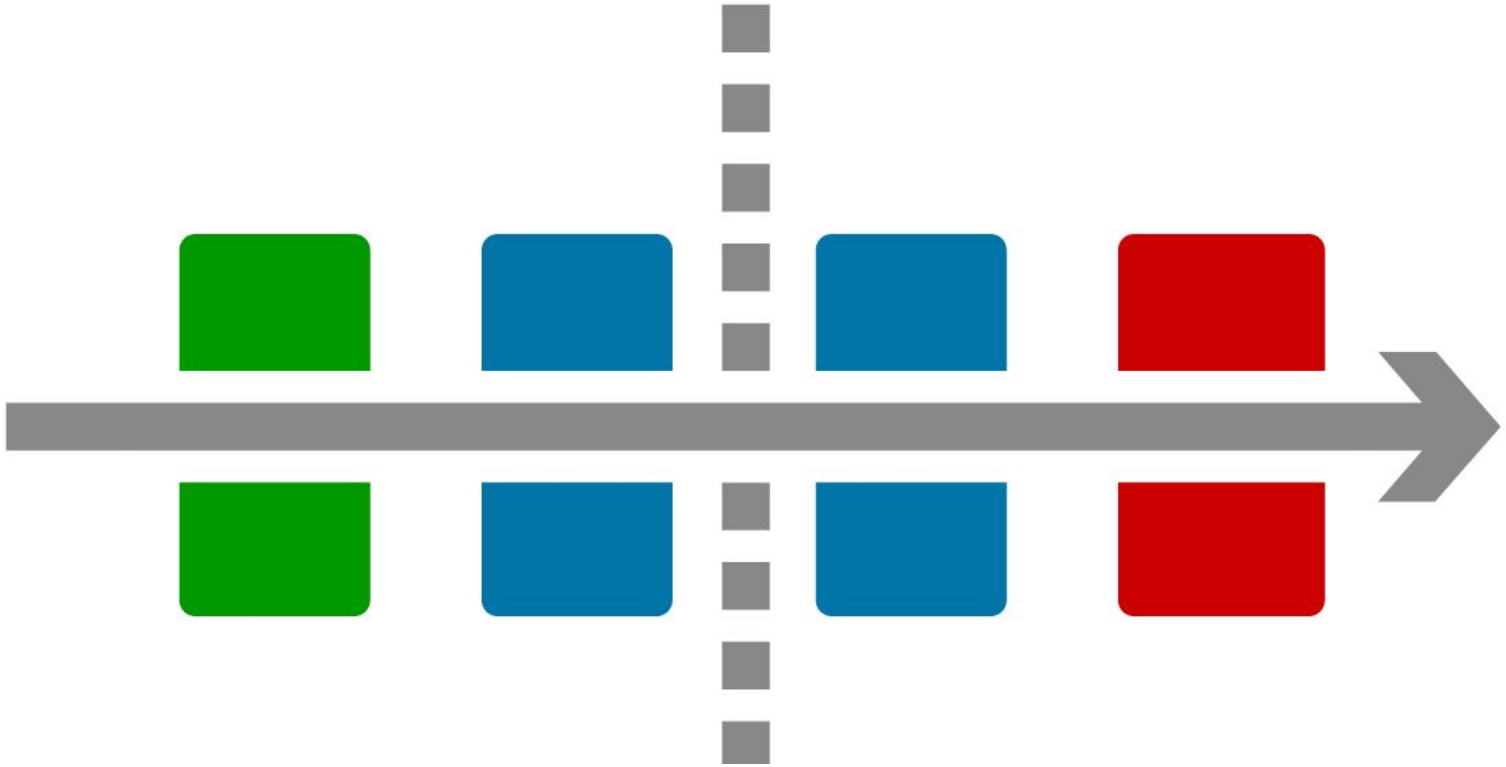
Оси



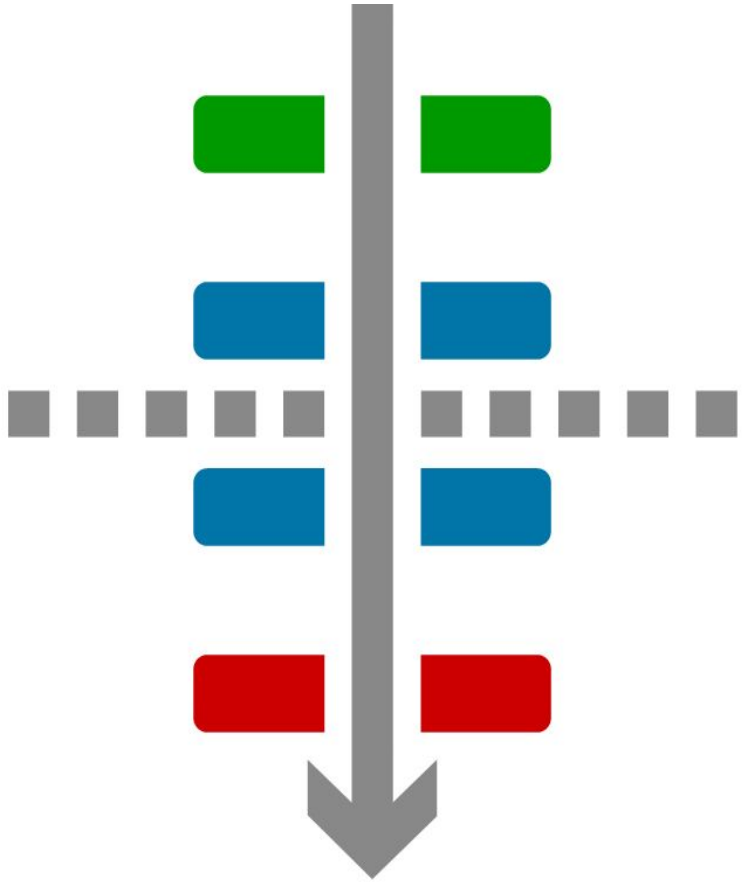
Оси


main size
cross size
cross start
main axis
1
2
flex item
flex item
cross end
flex container
main
start
cross axis
main
end

flex-direction: row;



flex-direction: column;

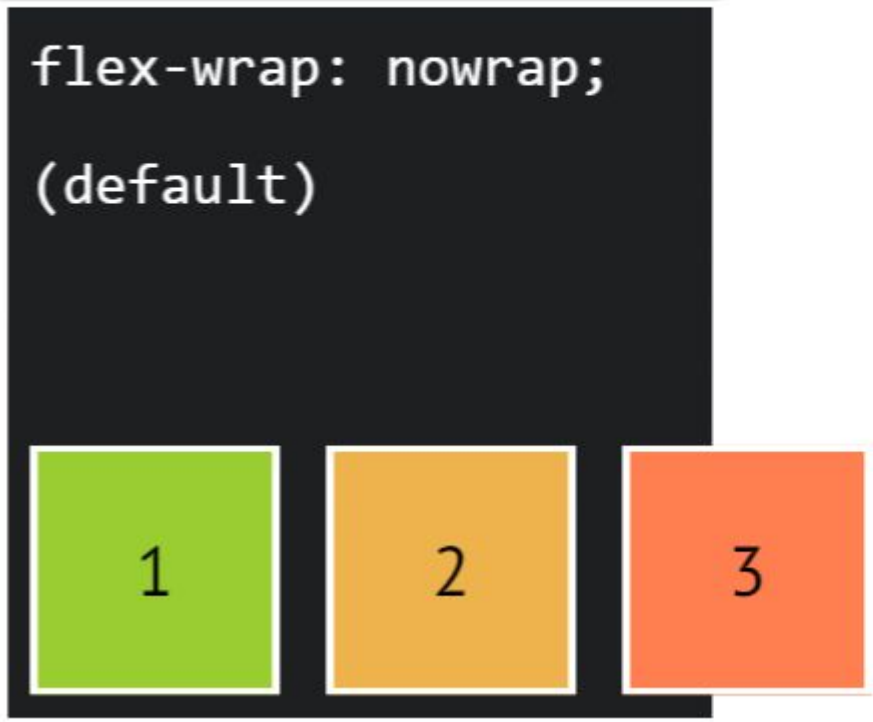


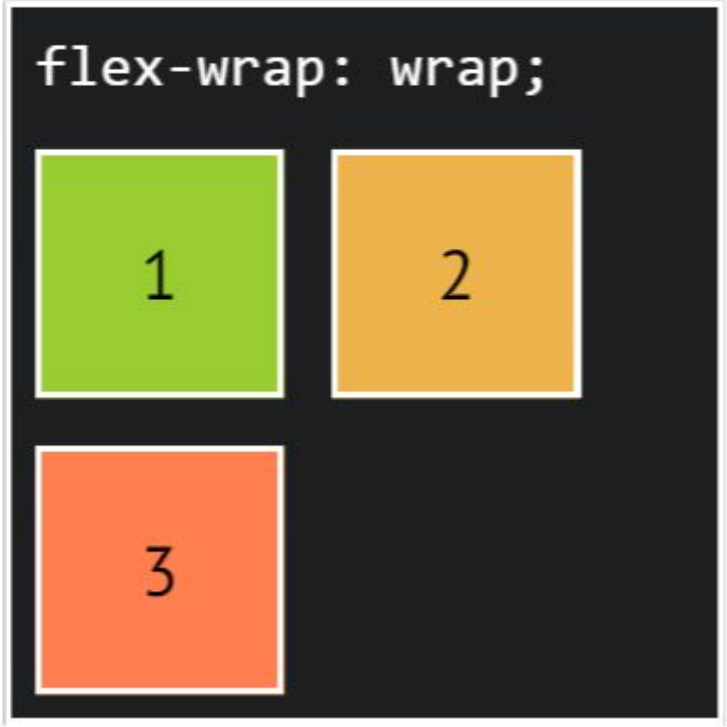
flex-direction vs wrap-reverse

- `flex-direction` - задает направление главной оси
- `flex-wrap: wrap-reverse;` - меняет направление поперечной оси

flex-wrap

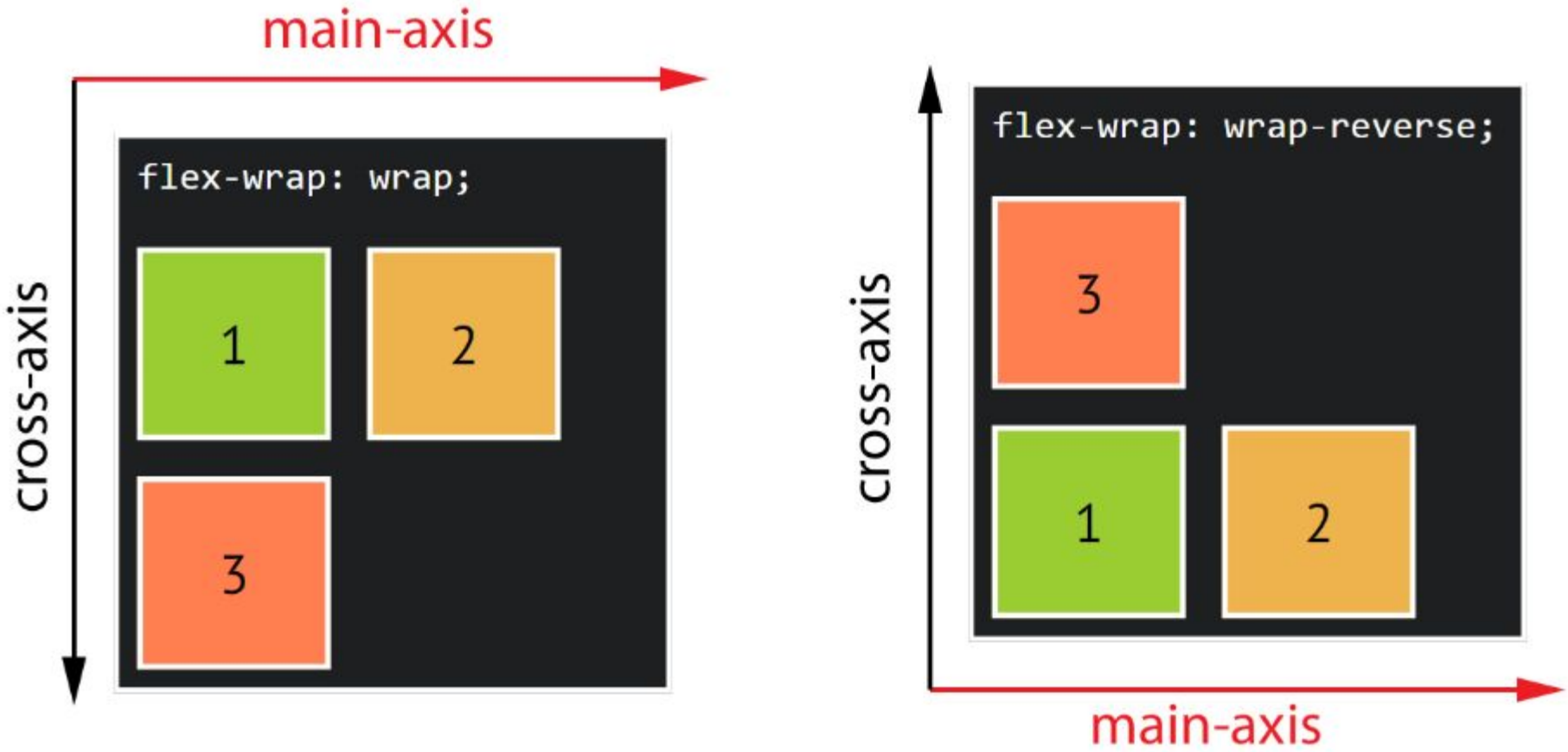
(свойство контейнера)





flex-wrap: wrap-reverse

меняет направление поперечной оси



flex-flow

Сокращенная форма `flex-wrap` и `flex-direction`

Плохо

```
01. .container{
02.         flex-direction: row-reverse;
03.         flex-wrap: wrap;
04. }
```

Хорошо

```
01. .container{
02.         flex-flow: row-reverse wrap;
03. }
```

Меняем направление по оси

```
.flex {
    height:640px;
      display:flex;
      background: #838283;
      flex-direction:row-reverse;
   }
```



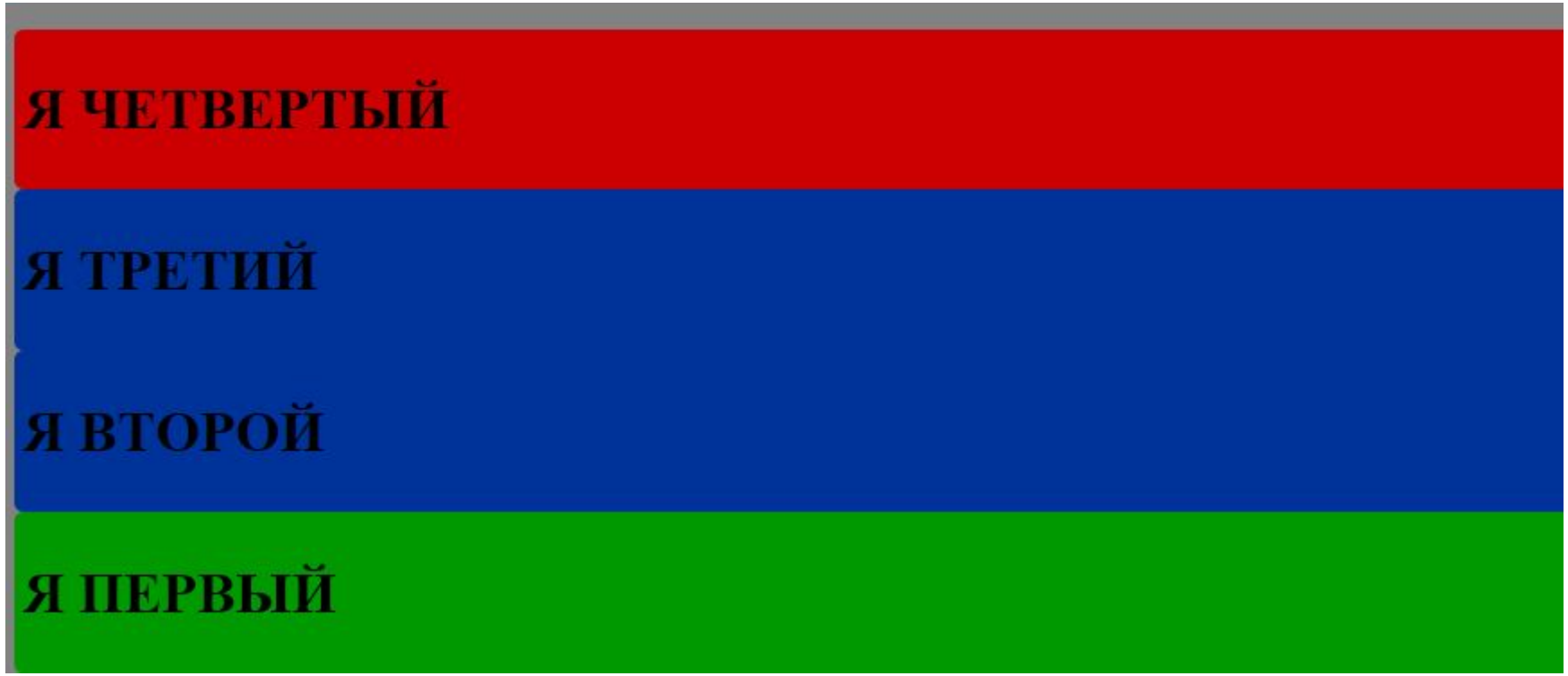
Поворачиваем саму ось

```
.flex {
    height:640px;
      display:flex;
      background: #838283;
      flex-direction:column;
   }
```



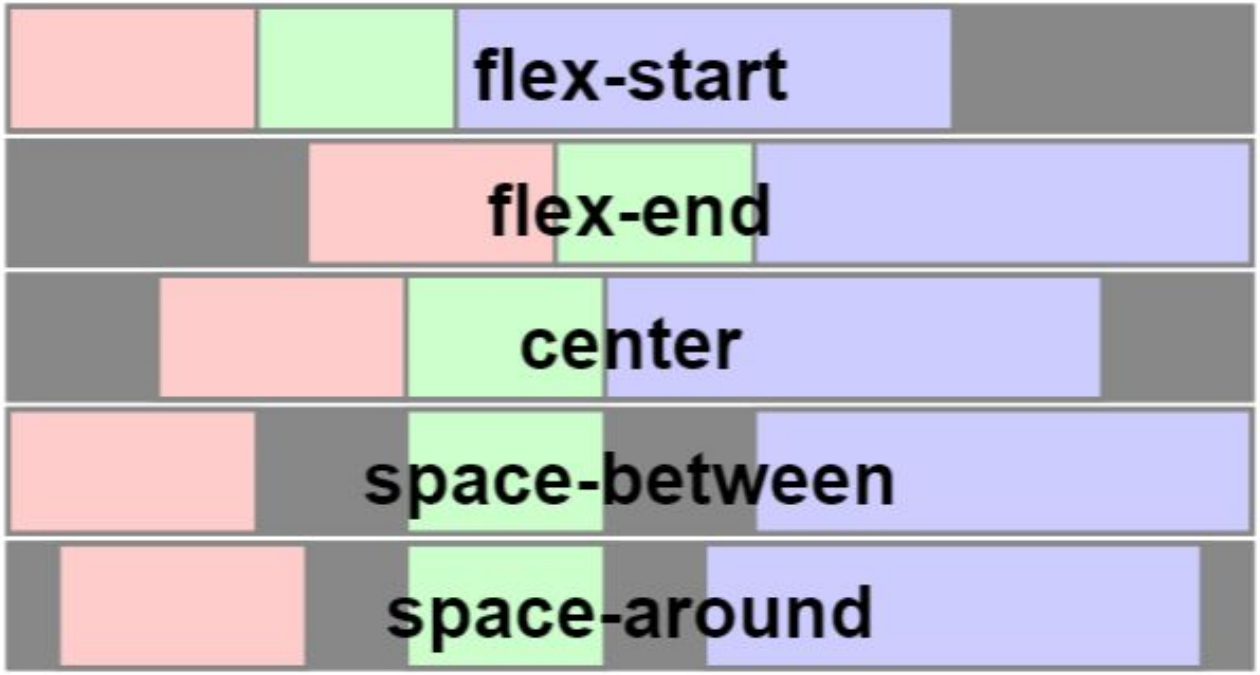
Меняем направление по оси

```
.flex {
    height:640px;
      display:flex;
      background: #838283;
      flex-direction:column-reverse;
   }
```



justify-content

Выравнивание по главной оси

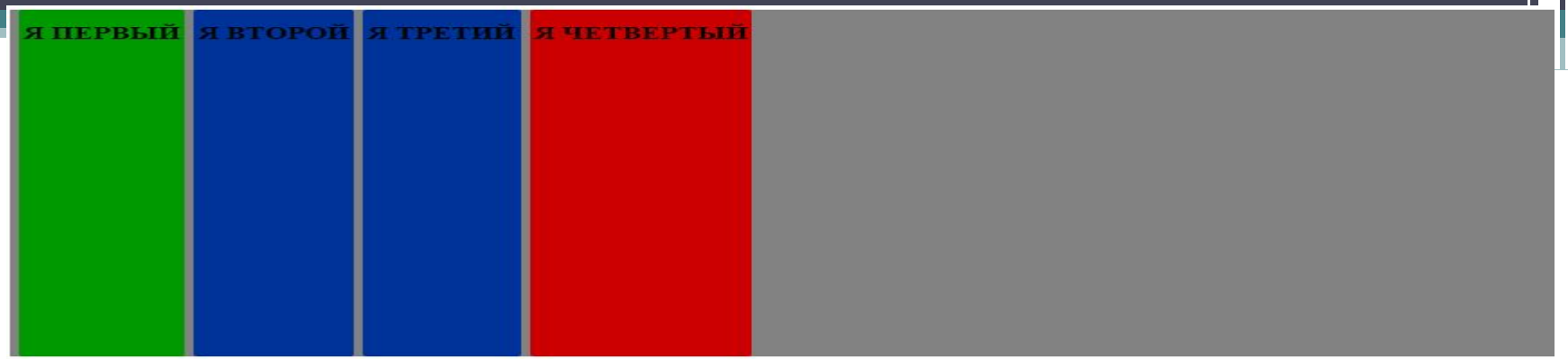


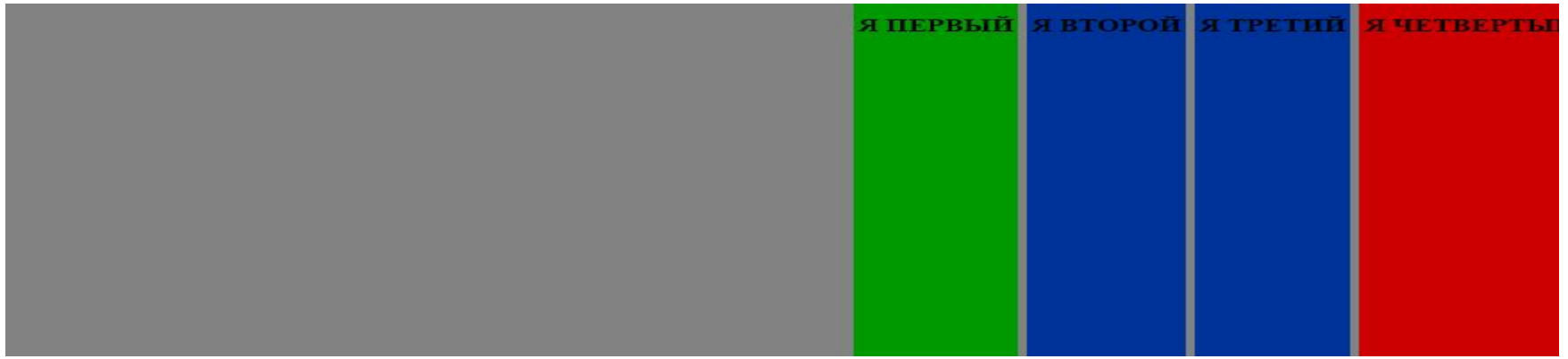
Пример

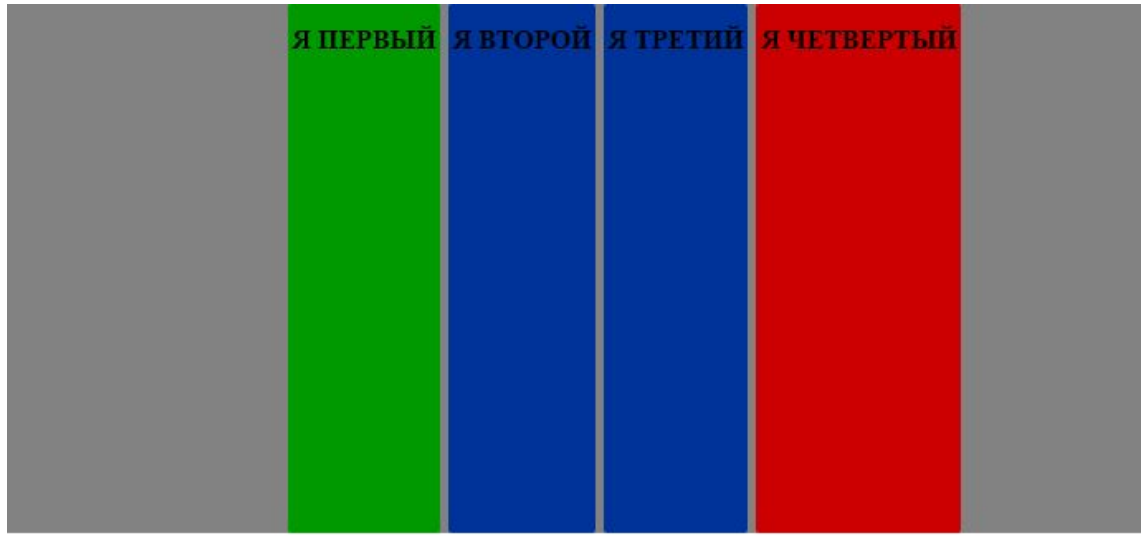
```
.flex {
    height:640px;
      display:flex;
      background: #838283;
     justify-content:flex-end;

.flex {  }
    height:640px;
      display:flex;
      background: #838283;
       justify-content:flex-end;
   }


    .flex {
        height:640px;
          display:flex;
          background: #838283;
           justify-content:center;
       }
```

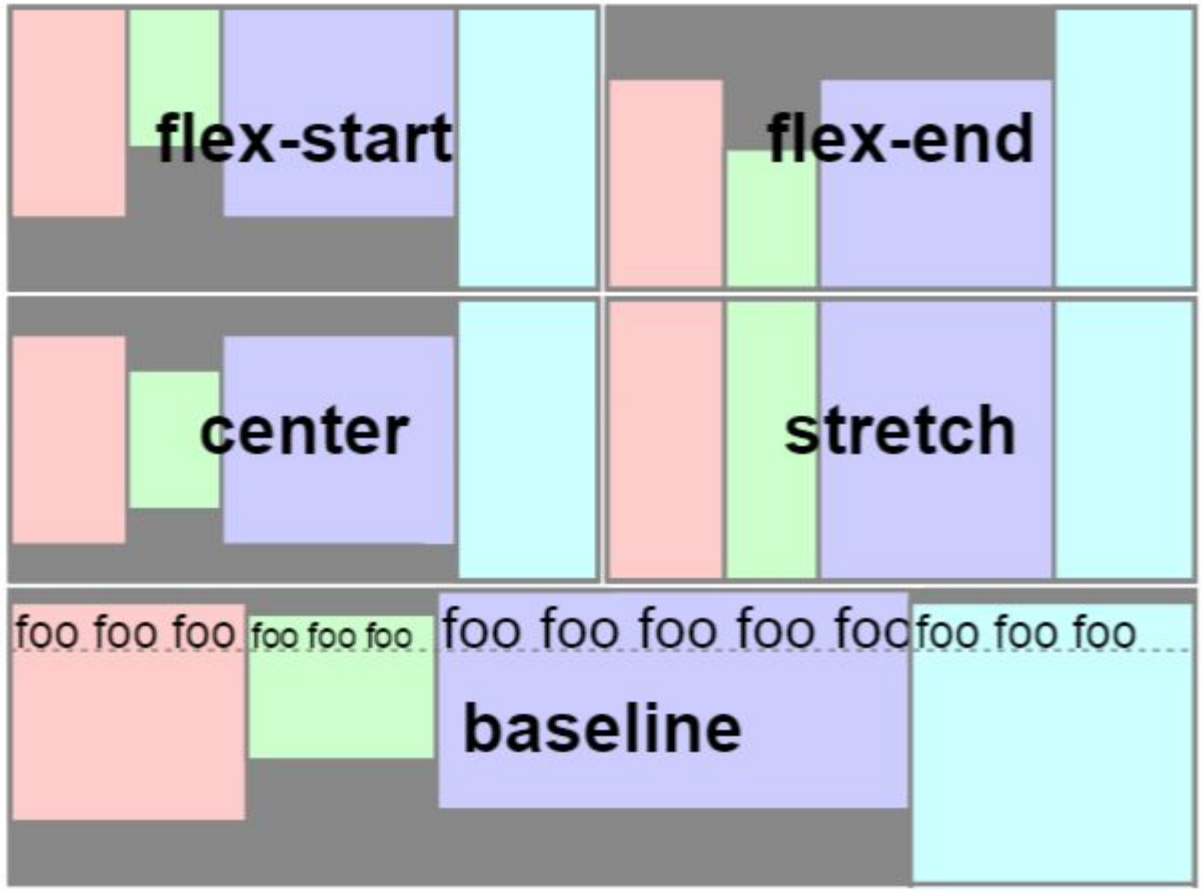






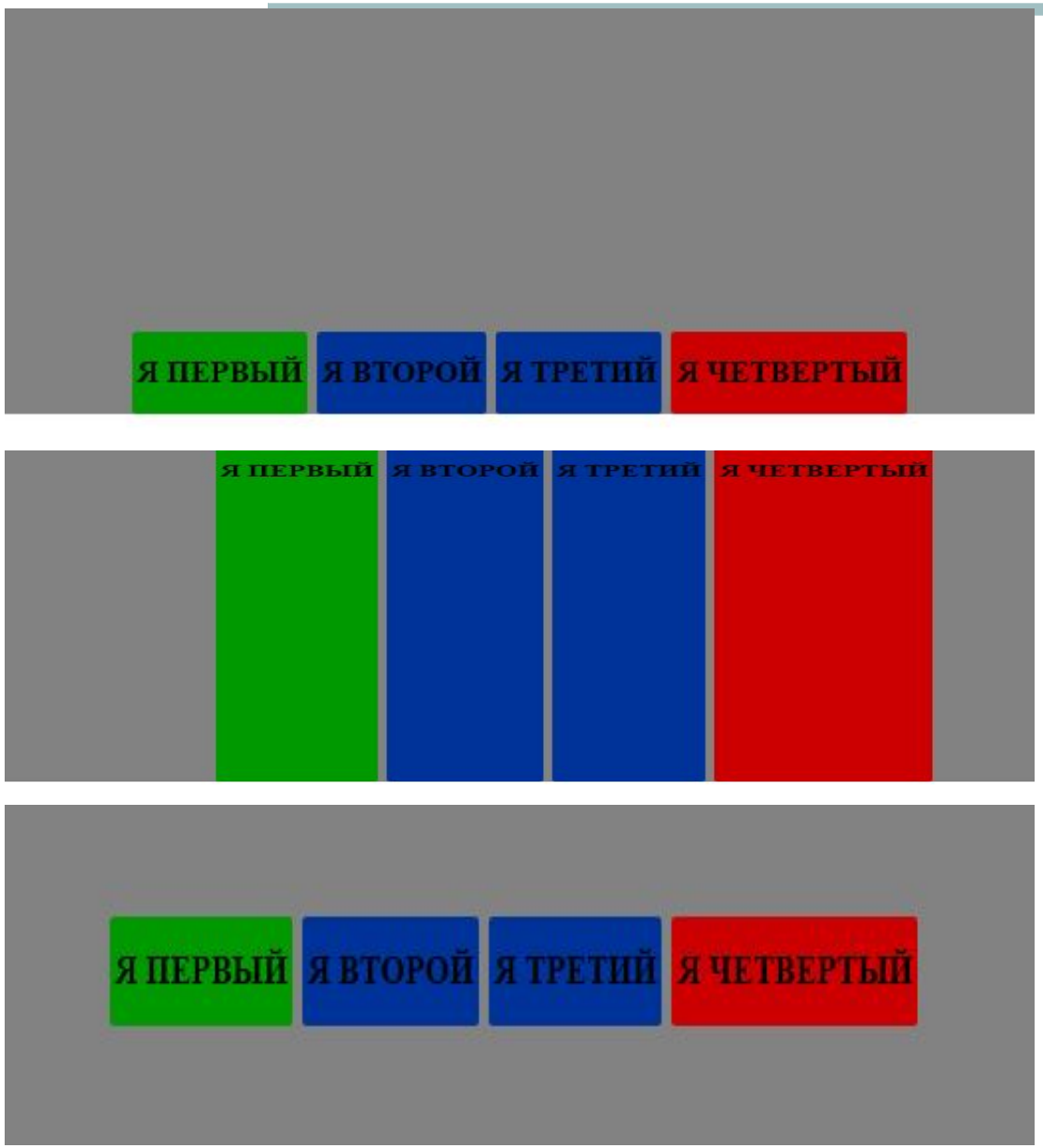
align-items

Выравнивание по поперечной оси



Пример

```
.flex {
    height:640px;
      display:flex;
      background: #838283;
       justify-content:center;
       align-items:flex-end;
   }
.flex {
    height:640px;
      display:flex;
      background: #838283;
       justify-content:center;
       align-items:stretch;
   } .flex {
        height:640px;
          display:flex;
          background: #838283;
           justify-content:center;
           align-items:center;
       }
```

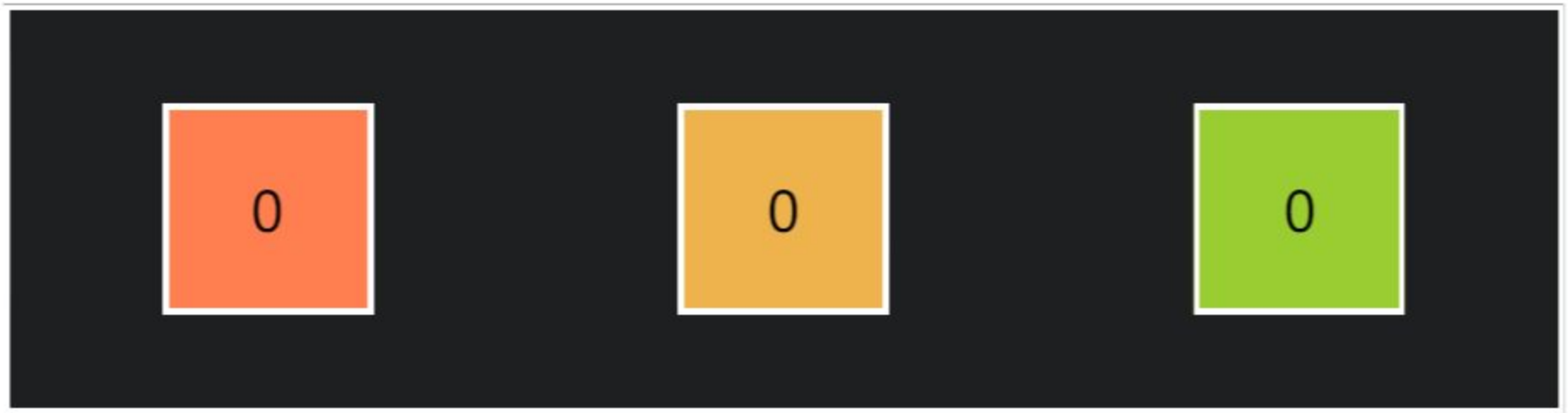


Перестановка

order

Применяется к: дочернему элементу

order: <integer>



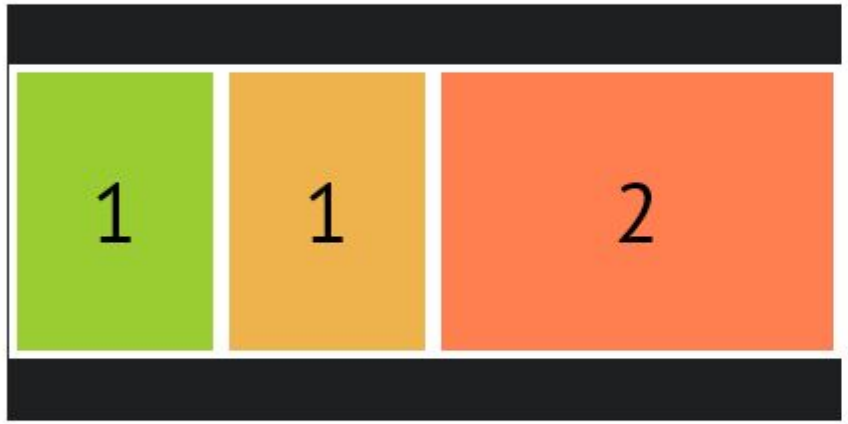
```
.flex div:first-child {
    background:#090;
       order:2;
    }
.flex div:last-child {
    background:#C00;
       order:1;
    }
.flex div:nth-child(2) {
    order:3;
    }
```



flex-grow- Применяется к: дочернему элементу. Определяет для flex-элемента возможность «вырастать» при необходимости.

flex-grow: <number> (по умолчанию 0)

flex-grow




```
01. .item{
02.             flex-grow: 1;
03. }
04. .item:last-of-type{
05.             flex-grow: 2;
06. }
```




```
.element-1
    {flex-grow: 1;}
.element-2
    {flex-grow: 4;}
```

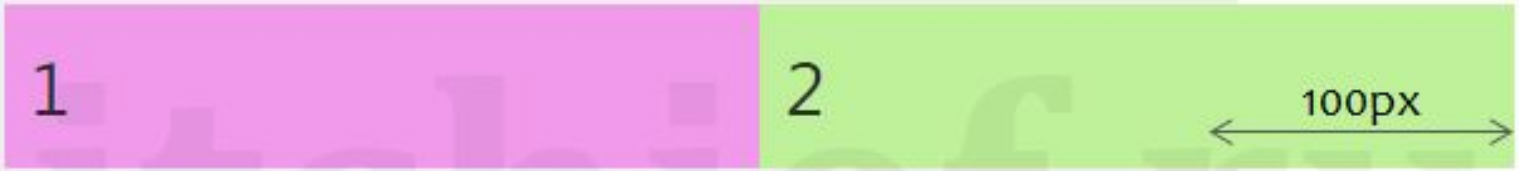
В этом примере, если flex-элементы расположены на одной линии и в ней есть свободное пространство (600×(1-0,8)=120px):

- к ширине элемента `.flex-container_element-1` добавится 1/5 часть этого пространства (120×1/5=24px);
- к ширине элемента `.flex-container_element-2` добавится 4/5 части этого пространства (120×4/5=96px).

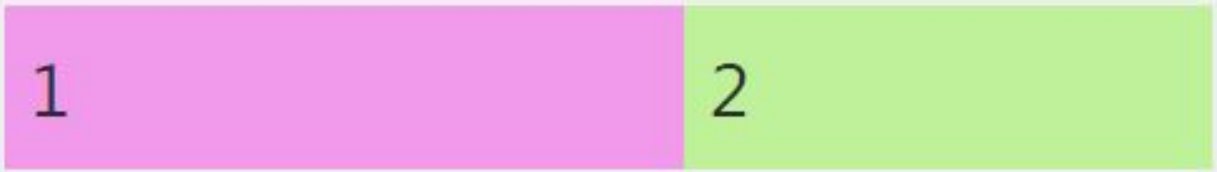
flex-shrink- Применяется к: дочернему элементу. Определяет для flex-элемента возможность «сжиматься» при необходимости. *По умолчанию (1)*

flex-shrink: <number> (по умолчанию 1)

flex-элементам запрещено уменьшаться (значение flex-shrink для первого и второго элемента равно 0)



flex-элементам разрешено уменьшаться (значение flex-shrink для первого flex-элемента равно 1, для второго – 3)




```
.element-1
    {flex-shrink: 1;}
.element-2
    {flex-shrink: 3;}
```

Ширина первого flex-элемента в данном случае составит 300 – 1/4*100= 275px.
Ширина второго flex-элемента в данном случае составит 300 – 3/4*100= 225px.

flex-basis- Применяется к: дочернему элементу. Определяет размер по умолчанию для элемента перед распределением пространства в контейнере. *По умолчанию (auto)*

content → width → flex-basis (ограниченный max|min-width)

Единственное, что реально важно, это конечный flex-basis. Лучшей практикой является использование просто flex-basis, вместо width и height.
Конечное значение flex-basis ограничено min-width и max-width элемента.
Если flex-basis не указан, то он опирается на свойство width этого же элемента. А если уже и width не указан, то flex-basis опирается ширину, посчитанную по контенту.

Сокращенная запись

flex: 0 1 auto;

Конечная ширина flex-элемента будет определяться в зависимости от значений CSS-свойств flex-grow и flex-shrink, которые установлены не только для этого элемента, но и для других flex-элементов этого flex-контейнера.

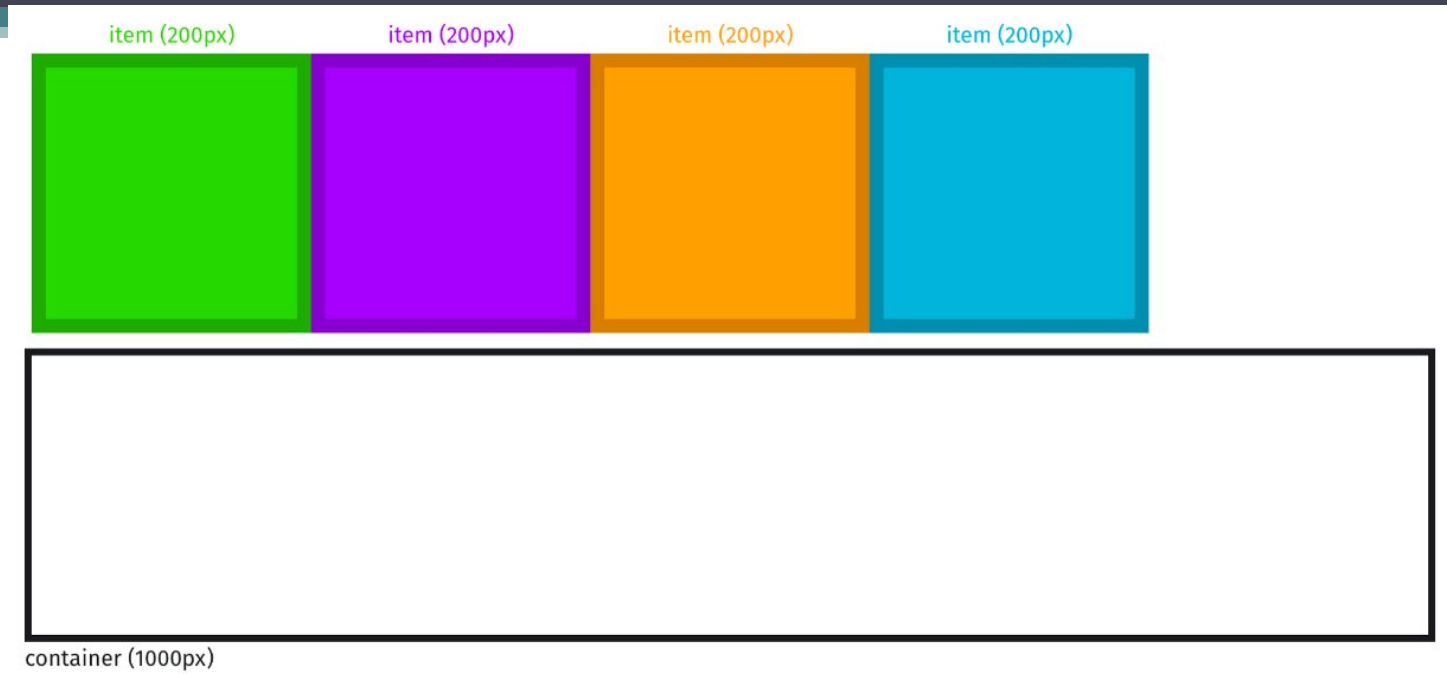


```
.container{
        width: 300px;
}
.item{
        flex-basis: 50px;
}
```




```
.container{
        width: 300px;
}
.item{
        flex-basis: 50px;
        flex-grow: 1;
}
```

```
item{
 width: 200px;
 flex-grow: 0;
 flex-basis: auto;
}
```

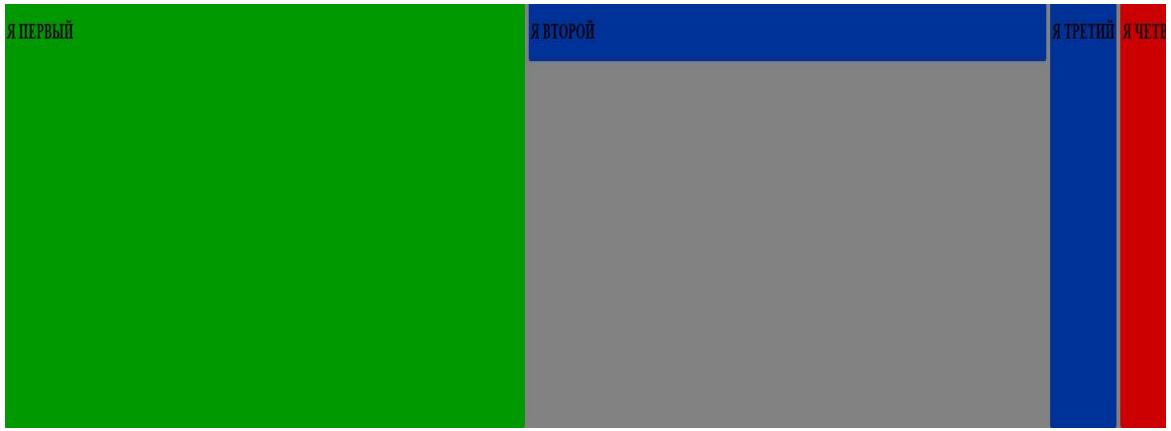



```
item{
 width: 30px;
 flex-grow: 1;
 flex-basis: 250px;
}
```



Align-self – выравнивание отдельного элемента (**Применяется к:** дочернему элементу)

```
.flex {
    height:640px;
      display:flex;
      background: #838283;
       justify-content:center;
       align-items:stretch;
   }
```




```
.flex div:nth-child(2) {
        flex-grow:1;
        align-self:flex-start;
      }
```

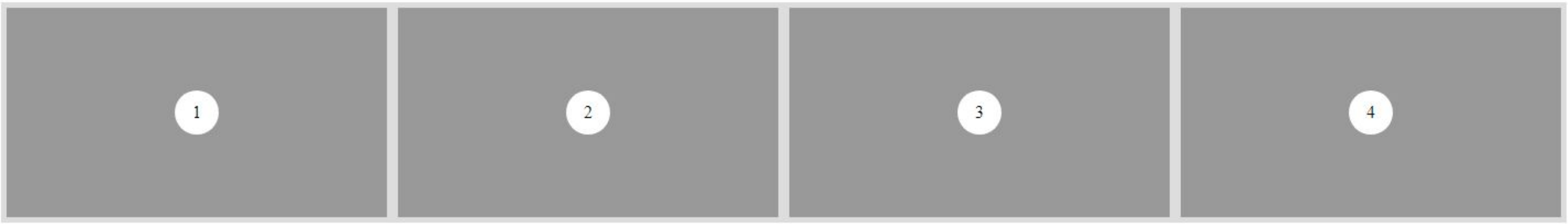
align-self: auto | flex-start | flex-end | center | baseline | stretch

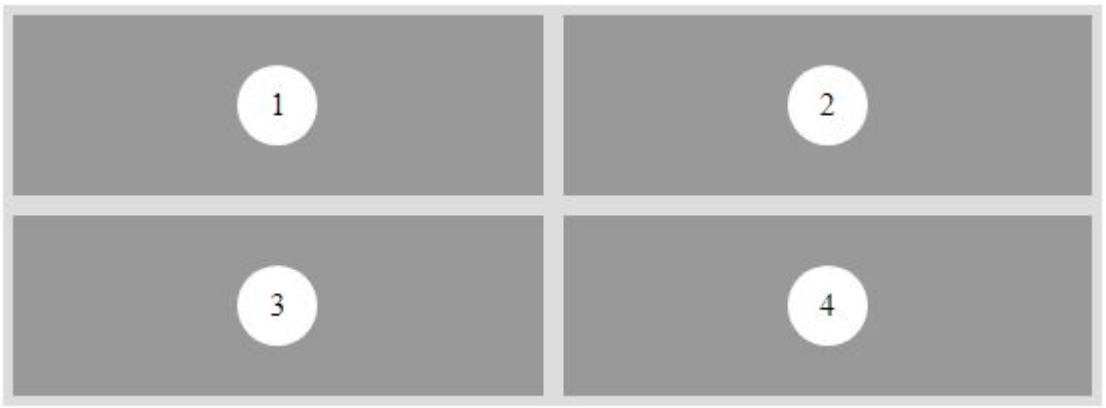
Поддержка - 97% браузеров с 2018 года

кроме IE 8,9

Задания

https://d-e-n.info/blog/templates/2019-05-31-internet-magazin-odezhdy.html


1
2
3
4


1
2
3
4

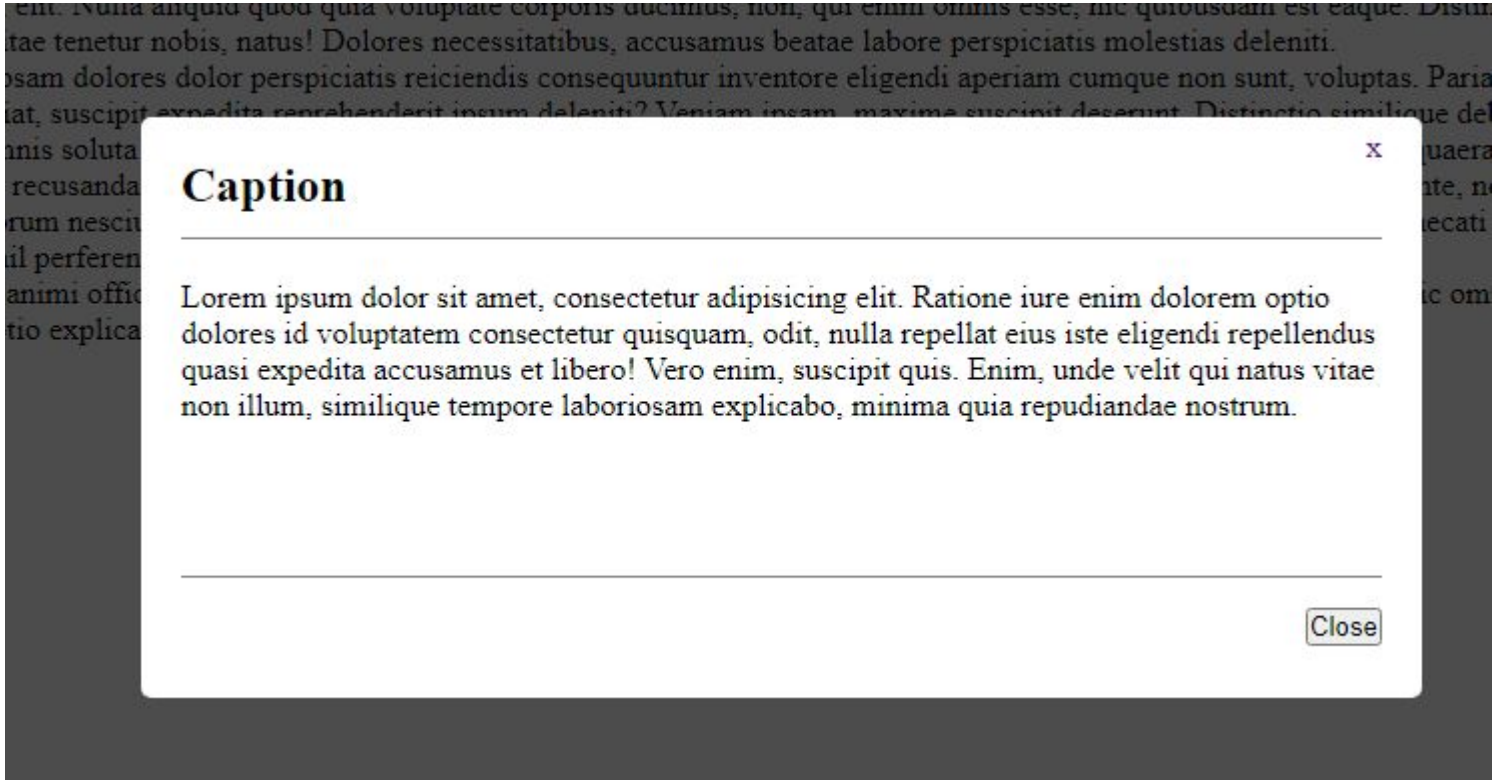


СТАРТ ПРОДАЖ

YEEZY 700 V3 ARZARETH

Открыта возможность предварительного заказа

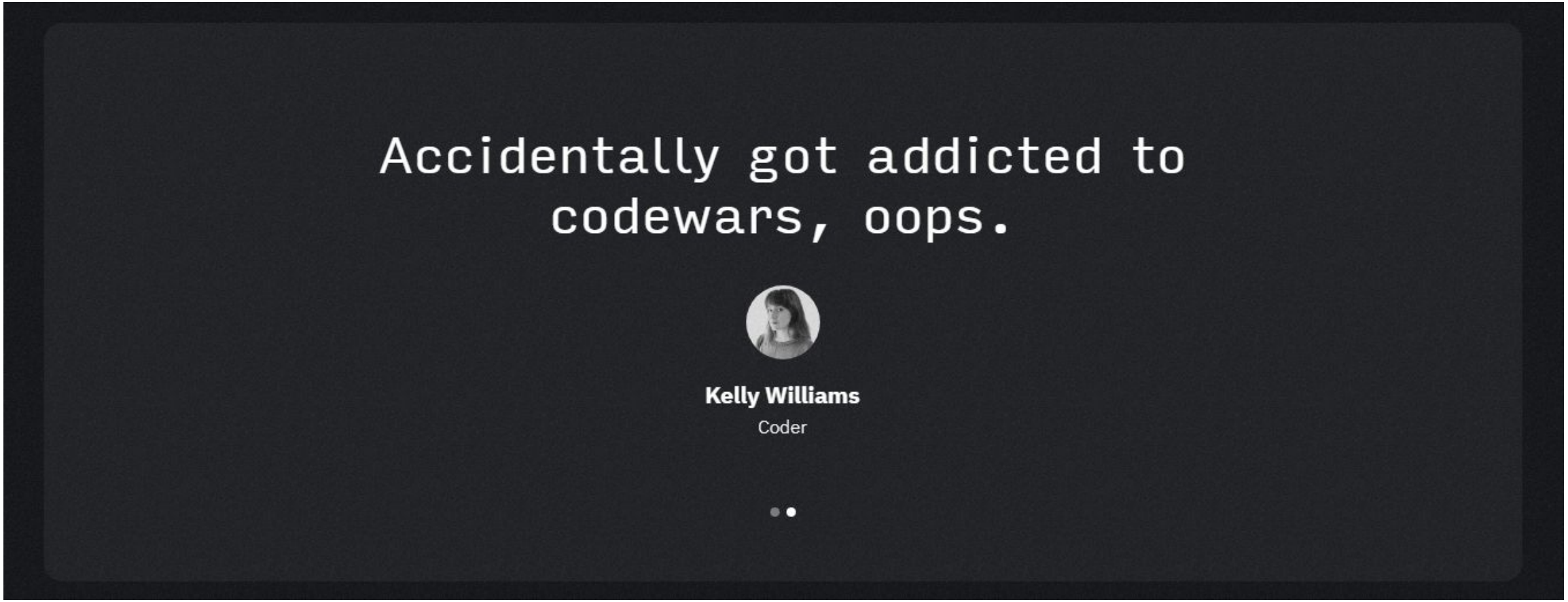
СМОТРЕТЬ


tae tenetur nobis, natus! Dolores necessitatibus, accusamus beatae labore perspiciatis molestias deleniti.
sam dolores dolor perspiciatis reiciendis consequuntur inventore eligendi aperiam cumque non sunt, voluptas. Paria
Caption
Lorem ipsum dolor sit amet, consectetur adipisicing elit. Ratione iure enim dolorem optio dolores id voluptatem consectetur quisquam, odit, nulla repellat eius iste eligendi repellendus quasi expedita accusamus et libero! Vero enim, suscipit quis. Enim, unde velit qui natus vitae non illum, similique tempore laboriosam explicabo, minima quia repudiandae nostrum.
Close

Первый

Второй

Третий



https://www.codewars.com/


Codewars for?
From beginner to expert and beyond...
Get new perspectives
Solve challenges then view how others solved the same challenge. Pickup new techniques from some of the most skilled developers in the world.
Learn new languages
Solve challenges in a language you are comfortable with, then do it in a language you want to improve with. Level up across different languages.
Compete with peers
Compete against your friends, colleagues, and the community at large. Allow competition to motivate you towards mastering your craft.

The latest

Insights from Codewars staff and community.



Career Insights

Developer Productivity: A guide to finding flow

Often used interchangeably with being "in the zone", flow blocks out the world and allows you to fully involve yourself with whatever you're doing. You feel increased confidence, motivation, productivity, and happiness all while being immersed in your work. Find out how to harness your flow.



Ellen Weaver
December 4, 2022

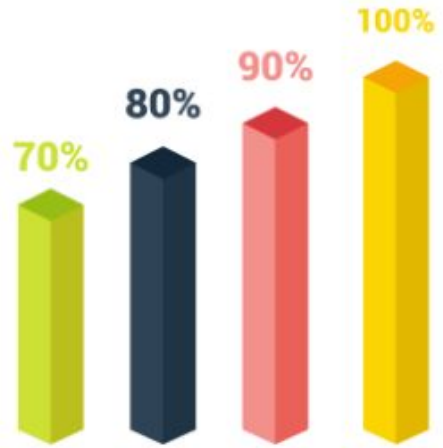
- ТЕКСТ 1
- ТЕКСТ 2
- ТЕКСТ 3
- ТЕКСТ 4
- ТЕКСТ 5

Заголовок

TRAFIC



LOADS



Заголовок

01/12/2020

1 Lorem ipsum dolor. Lorem ipsum. 1/12/2020
2 Lorem ipsum dolor. Lorem ipsum. 8/07/2020
3 Lorem ipsum dolor. Lorem ipsum. 05/9/2020
4 Lorem ipsum dolor. Lorem ipsum. 01/11/2020

Читать

- Теория (https://jonibologna.com/blog/flexbox-cheatsheet)
- Поддержка Flexbox в браузерах
- Таблица соответствия F09 и F11
- Таблица соответствия F09 и F11 для IE10
- CSS3 Flexplorer
- Использование Flexbox от Mozilla
- Руководство по Flexbox от Microsoft

Задание

